PHASE 5

Governed Agentic Security Platform on GKE

Plan Mode Project Summary and Implementation Guide

Google Cloud | GKE | Vertex AI / Gemini | OPA | Cloud KMS | MCP | GitHub Actions

Document purpose

This document adapts the Cursor AI Plan Mode workflow to the complete Phase 5 repository. It captures clarified scope, research findings, file organization, implementation tasks, dependencies, risks, testing, success criteria, review gates, and the controlled deployment sequence.

Project type	Student portfolio lab with production-minded architecture and security controls
Current status	Configuration-ready; live Google Cloud, GKE, KMS, policy, and release validation still required
Complexity	9/10 - multi-service, event-driven, policy-governed Kubernetes platform
Safety posture	Fail-closed; remediation and external reporting disabled by default
Prepared by	T.I.Q.S.

Prepared for technical review, recruiter discussion, and guided implementation

Phase 5 Network Architecture

(Coming Soon)

Figure 1. Governed multi-agent security workflow, trust boundaries, event topics, and supporting Google Cloud services.

Document roadmap

Section	Purpose
Phase 1	Clarification - scope, requirements, constraints, goals, and success criteria
Phase 2	Research - current-state assessment, component mapping, and design patterns
Phase 3	Plan generation - structure, checklist, dependencies, risk, testing, and next steps
Phase 4	Review - formal approval gate before controlled execution
Phase 5	Implementation - deployment sequence, operational handoff, and validation
Appendix A	Major component summary and enterprise relevance
Appendix B	Reusable Cursor AI Plan Mode prompt for future changes

Plan Mode rule

No live remediation, production deployment, or irreversible infrastructure change should occur until the plan is reviewed, the required tests pass, and an authorized reviewer approves the action.

Phase 1: Clarification

The original Plan Mode template begins by clarifying scope, technology, constraints, goals, dependencies, and blockers. For this project, those questions have been resolved as follows.

Clarification area	Resolved requirement
Project scope	Build a governed, event-driven security automation platform on GKE. AI assists analysis but does not independently authorize or execute infrastructure changes.
Cloud and region	Google Cloud project class-6-5-tiqs; us-central1 / us-central1-a; GKE cluster vertex-agent-lab.
Core stack	Terraform, GKE, Workload Identity, Pub/Sub, Vertex AI/Gemini, OPA/Rego, Cloud KMS, Firestore, BigQuery, Secret Manager, Cloud Logging, Cloud Monitoring, Docker, GitHub Actions, Trivy, Syft, and Cosign.
Primary security objective	Enforce deterministic authorization, signed human approval, least privilege, mTLS, replay prevention, immutable releases, and durable evidence.
Target workload	Demo application and PostgreSQL in app01; the only controlled action is against explicitly allowlisted Kubernetes resources.
Current repository state	Canonical Phase 5 files are present; historical Phase 1-4 workflows and retired agents were removed from the minimal structure.
Default execution posture	Governance and remediation remain disabled until explicitly changed to controlled after review and validation.
Success definition	End-to-end findings reach an incident report, sensitive actions require valid governance and approval evidence, and no bypass path exists around MCP, mTLS, policy, or RBAC.

Clarification area	Resolved requirement
Known blocker	Live validation requires Google Cloud credentials, a reachable GKE control plane, a configured backend bucket, and installed policy/release tooling.

Clarification checklist

- [x] Scope and in-scope services are documented.
- [x] Google Cloud project, region, zone, and cluster naming are defined.
- [x] The target Kubernetes namespaces and allowlisted workload are defined.
- [x] Fail-closed defaults are documented.
- [ ] Cloud KMS reviewer identities must be replaced with real Google identities.
- [ ] The administrator workstation CIDR must be set in the ignored terraform.tfvars.
- [ ] The production GitHub environment reviewer must be configured.

Phase 2: Research

The research phase identifies the active files, dependency chain, architectural patterns, and likely failure points before implementation or redeployment.

Current-state findings

Area	Finding
Architecture	Multi-stage Pub/Sub pipeline with isolated analysis, governance, approval, remediation, and reporting roles.
Identity	Dedicated Kubernetes and Google service accounts mapped through Workload Identity; GitHub uses OIDC federation.
Governance	OPA/Rego policy produces DENY, NO_ACTION, PERMIT, or REQUIRE_APPROVAL.
Approval	Cloud KMS asymmetric signing binds the reviewer to a specific request, decision, expiry, and key version.
State	Firestore stores correlation windows, approval consumption, and execution idempotency state.
Execution boundary	Remediation Agent uses an mTLS client certificate to reach nginx MCP Gateway; MCP Server holds scoped Kubernetes RBAC.
Evidence	Cloud Logging and BigQuery retain governance, approval, remediation, and MCP evidence.
Reliability	Pub/Sub retry/DLQ, HPA, PDB, probes, resource limits, and Managed Prometheus metrics.
Supply chain	GitHub Actions, Trivy, Syft, Cosign, immutable digests, SBOMs, signatures, and attestations.

Dependency chain

```
Terraform foundation
  -> GKE + Workload Identity + Artifact Registry
  -> Pub/Sub + Firestore + KMS + BigQuery + monitoring
  -> Container image build and release evidence
  -> Namespaces + service accounts + RBAC
  -> MCP certificates and gateway
  -> Demo workload + scanners
  -> Event, analysis, governance, approval, remediation, reporting agents
  -> NetworkPolicies + PodMonitoring + HPA + PDB
  -> Smoke tests + signed approval test + controlled execution review
```

Relevant repository areas

Path	Responsibility
.github/workflows	CI validation and governed release workflow
docker	One build definition for each runtime image
docs	Runbook, architecture, and governance concepts
manifests	Kubernetes workloads, policies, reliability, observability, and deployment RBAC
policy	Conftest and OPA/Rego policies
python	Agent services, shared runtime utilities, tests, and dependencies
scripts	Build, deploy, validate, test, approval, certificate, and release automation
terraform	GCP foundation, IAM, messaging, state, governance, evidence, and monitoring

Phase 3: Plan generation

Implementation status legend

[x] means implemented in the repository. [ ] means environment-specific, live-cloud, or approval work remains.

1. Overview and implementation objective

Complete and validate a portfolio-grade Phase 5 platform that demonstrates responsible agentic AI integration on Kubernetes. The implementation must prove that analysis, authorization, approval, execution, and evidence are separate controls and that the AI model cannot bypass deterministic governance.

2. Canonical file structure

```
agentic/
├── .github/workflows/          # Phase 5 CI and governed release
├── docker/                     # Runtime image definitions
├── docs/                       # Runbook, architecture, and concepts
├── examples/                   # Non-runtime classroom examples
├── helm/falco/                 # Falco values
├── manifests/                  # Kubernetes workloads and controls
│   ├── network-policies/
│   ├── observability/
│   ├── rbac/
│   └── reliability/
├── policy/                     # Conftest and OPA/Rego
├── python/                     # Services, shared modules, tests
├── scripts/                    # Build, deploy, approval, test, release
├── terraform/                  # Numbered Phase 5 GCP infrastructure
└── README.md
```

3. Detailed implementation steps

Repository hygiene

- [x] Keep only Phase 5 workflows and canonical runtime files.
- [x] Keep .terraform.lock.hcl and exclude .terraform, state, local tfvars, credentials, certificates, and release evidence.
- [x] Replace committed PostgreSQL credentials with a generated Secret workflow.

- [ ] Remove orphaned or retired Dockerfiles and historical agents.

Terraform and Google Cloud foundation

- [ ] Configure the GCS backend and sanitized terraform.tfvars.
- [ ] Provision VPC, subnet, GKE, node pool, Artifact Registry, and APIs.
- [ ] Provision Pub/Sub topics, subscriptions, retry policies, and DLQ.
- [ ] Provision agent identities and Workload Identity bindings.
- [ ] Provision Firestore, KMS, BigQuery evidence, Secret Manager, and monitoring.
- [ ] Review terraform plan before apply.

Build and supply chain

- [ ] Build the eleven Phase 5 images.
- [ ] Scan repository and images for vulnerabilities, secrets, and misconfiguration.
- [ ] Generate SPDX SBOMs.
- [ ] Sign immutable digests and attach SBOM attestations.
- [ ] Verify signature identity and attestation issuer.

Kubernetes platform

- [x] Create seven namespaces and labels.
- [x] Apply service accounts and least-privilege RBAC.
- [ ] Generate MCP server/client CAs and Kubernetes Secrets.
- [ ] Deploy MCP Gateway, MCP Server, PostgreSQL, and demo application.
- [ ] Deploy Event Aggregator and agent workloads.
- [ ] Apply NetworkPolicies, HPA, PDB, PodMonitoring, quotas, and limits.

Telemetry and AI analysis

- [ ] Install Falco and deploy Trivy/Prowler scanner CronJobs.
- [ ] Verify raw-security-events and security-findings ingestion.
- [ ] Verify Observer and Correlation Agent outputs.
- [ ] Verify Gemini-assisted analysis produces normalized analyzed-incidents.

Governance and approval

- [x] Mount and test remediation.rego.
- [ ] Verify DENY, NO_ACTION, PERMIT, and REQUIRE_APPROVAL behavior.
- [ ] Generate a Cloud KMS-signed approval decision.
- [ ] Verify request hash, expiry, key version, reviewer, and replay protection.

Remediation and evidence

- [x] Keep execution mode disabled during dry-run tests.
- [ ] Verify Remediation Agent independently validates governance evidence.
- [ ] Verify mTLS trust and MCP target allowlists.
- [ ] Verify Firestore prevents duplicate execution.
- [ ] Verify remediation result, incident report, logs, metrics, and BigQuery evidence.

Controlled activation

- [ ] Obtain formal review approval.
- [ ] Change governance and execution modes to controlled.
- [ ] Run one allowlisted, reversible controlled action.
- [ ] Confirm evidence and rollback behavior.
- [ ] Return to disabled mode after the demonstration if continuous automation is not required.

4. Dependencies and prerequisites

Dependency	Purpose
Terraform >= 1.10	Infrastructure initialization, validation, plan, apply, and destroy
gcloud and kubectl	Authentication, Artifact Registry, GKE credentials, Pub/Sub, KMS, and Kubernetes operations
Docker + Buildx	Local multi-image build workflow
Python 3.11 + pytest	Agent runtime, helpers, tests, and secret generation
OPA + Conftest	Governance policy and Kubernetes policy validation
OpenSSL	MCP server and client certificate generation
Helm	Falco installation
Trivy, Syft, Cosign	Security scanning, SBOMs, signatures, and attestations
GitHub repository variables	WIF provider, release service account, project, cluster name, and cluster location
Authorized reviewers	Cloud KMS signer IAM and protected GitHub production environment

5. Potential issues and mitigations

Risk	Impact	Mitigation
GKE control plane is unreachable	Deployment, security, or workflow failure	Update the workstation /32 CIDR in the ignored terraform.tfvars and reapply.
VS Code cannot resolve every GitHub Action	Deployment, security, or workflow failure	Treat it as an editor resolver/authentication/network issue; validate with GitHub-hosted runs or actionlint.
Existing default Firestore database conflicts with Terraform creation	Deployment, security, or workflow failure	Set manage_firestore_database=false and manage only the required fields and TTL configuration.
Artifact Registry cannot be destroyed	Deployment, security, or workflow failure	Delete all image packages and versions before terraform destroy.
Pub/Sub stage repeatedly fails	Deployment, security, or workflow failure	Inspect the consumer logs, IAM, message schema, and agent-events-dlq-sub.
Approval signature is rejected	Deployment, security, or workflow failure	Verify canonical JSON, request hash, exact key version, reviewer, expiry, and decision binding.
NetworkPolicy blocks a required service	Deployment, security, or workflow failure	Verify DNS, metadata server, Google API, MCP, Prometheus, and app01 PostgreSQL paths.
Duplicate execution	Deployment, security, or workflow failure	Use Firestore transactions and idempotency keys before invoking MCP.
Model response is malformed or unsafe	Deployment, security, or workflow failure	Reject malformed output; do not allow AI output to bypass OPA or deterministic application checks.
Certificate trust failure	Deployment, security, or workflow failure	Rotate mTLS material, verify SANs and CA mounts, and restart gateway, server, and remediation workloads.

6. Testing strategy

Test layer	Method	Acceptance result
Static code	Python compileall and pytest	All tests pass with no import or syntax errors
YAML	Parse Kubernetes and workflow YAML	All documents parse successfully
Shell	bash -n against all scripts	No syntax errors
Terraform	fmt, init -backend=false, validate, reviewed plan	Configuration validates and plan matches intended resources
OPA	opa test policy/governance -v	All remediation policy tests pass
Conftest	Evaluate active Kubernetes manifests	No policy violations
Supply chain	Trivy, Syft, Cosign sign/verify	No gated findings; SBOM and verified signature exist for each digest
Kubernetes	Rollout status, probes, RBAC, NetworkPolicy checks	All workloads ready and only intended network/access paths are allowed
Event pipeline	Publish correlated findings and inspect final report	End-to-end incident report received
Governance	Dry-run REQUIRE_APPROVAL scenario	No unauthorized remediation occurs
Approval	KMS sign and Approval Agent verification	Valid approval accepted; expired/replayed/modified approvals rejected
Controlled action	One allowlisted reversible action	Action succeeds once, evidence is complete, and rollback is available

7. Success criteria

- [ ] Every agent uses a dedicated identity and only required permissions.
- [ ] All event stages have retry and dead-letter handling.
- [ ] Gemini assists analysis but cannot authorize or directly execute Kubernetes actions.
- [ ] OPA produces deterministic decisions and policy version evidence.
- [ ] Critical or policy-selected actions require a valid signed human approval.
- [ ] Approval replay, expiry, request mutation, and duplicate execution are rejected.
- [ ] Only the Remediation Agent can authenticate to the MCP gateway.
- [ ] MCP Server RBAC is limited to the allowlisted app01 operations.
- [ ] Every release has vulnerability results, an SBOM, a verified signature, and immutable digest evidence.
- [ ] Cloud Logging, metrics, Firestore, Pub/Sub, and BigQuery provide traceable operational evidence.
- [ ] The platform can be cleanly deployed, tested, disabled, rolled back, and destroyed.

8. Next steps

1. Replace placeholder reviewer and administrator network values in local configuration.
2. Run phase5-validate.sh with Terraform, OPA, and Conftest installed.
3. Review and apply the Terraform plan.
4. Build and release the image set with SBOMs and Cosign evidence.
5. Deploy Phase 5 with remediation disabled.
6. Run metrics, governance, pipeline, approval, and evidence tests.
7. Complete a formal review before enabling controlled mode.
8. Record screenshots, logs, plan output, and evidence for the portfolio demonstration.

Time estimate and complexity

Workstream	Estimate
Repository cleanup and local validation	4-8 hours
Terraform review and cloud provisioning	6-12 hours
Image build and release evidence	4-8 hours
Kubernetes deployment and troubleshooting	8-16 hours
Governance, approval, and controlled-action testing	8-16 hours
Documentation and portfolio evidence	4-8 hours
Student-lab total	Approximately 34-68 focused hours
Enterprise hardening beyond this lab	Approximately 12-20 weeks for a small cross-functional team

Complexity rating: 9/10

The platform spans cloud infrastructure, Kubernetes networking and identity, event-driven services, AI analysis, policy enforcement, cryptographic approvals, observability, evidence retention, and secure release engineering.

Security, compliance, and audit considerations

- Separation of analysis, authorization, approval, and execution responsibilities.
- Least-privilege IAM and namespace-scoped Kubernetes RBAC.
- Signed reviewer decisions with an auditable KMS key version.
- Evidence retention and data-classification review for logs, prompts, findings, and reports.
- Secret values added outside Terraform and never committed to the repository.
- Admission-time trust enforcement should be added for a Fortune 500 deployment.
- Formal threat modeling, red teaming, DR, SLOs, and incident-response ownership are required for production.

Multi-environment support

Environment	Required differences
Development	Low-cost cluster, synthetic telemetry, automation disabled, short retention, developer reviewer.
Staging	Production-like policy, KMS, WIF, scanners, evidence, and approval tests; controlled actions against non-production workloads.
Production	Private/regional architecture, protected release environment, independent reviewers, admission enforcement, managed PKI, DR, SLOs, and formal change control.

Rollback strategy

- Keep automation and execution disabled until the review gate passes.
- Deploy immutable image digests so each workload can be rolled back to a known release.
- Use kubectl rollout undo for application-level rollback where appropriate.
- Revert governance and remediation modes to disabled immediately after an unexpected action.
- Revoke or rotate the MCP client certificate if the execution identity is compromised.
- Disable a KMS key version and remove signer IAM after approval compromise.

- Use Terraform plans and state-aware changes for infrastructure rollback; do not manually edit managed cloud resources without reconciliation.

Phase 4: Review

The implementation may proceed to live controlled execution only after an authorized reviewer accepts the plan and the evidence below is complete.

- [ ] Repository contains only approved active files
- [ ] No plaintext secrets, keys, state, tfvars, or generated credentials are committed
- [ ] Terraform plan reviewed
- [ ] Python, YAML, Bash, Terraform, OPA, and Conftest validation passed
- [ ] Image scan, SBOM, signature, and attestation verification passed
- [ ] All Deployments are ready and health checks pass
- [ ] NetworkPolicy and RBAC negative tests pass
- [ ] Governance dry-run returns REQUIRE_APPROVAL without execution
- [ ] Cloud KMS approval is signed and verified
- [ ] Replay and duplicate-execution tests pass
- [ ] BigQuery and logging evidence is complete
- [ ] Rollback procedure was tested

Approval gate

Reviewer decision: [ ] Approve controlled implementation [ ] Request changes [ ] Reject.
Reviewer: _____ Date: _____ Evidence reference: _____

Phase 5: Implementation

Deployment sequence

```
# Validate
chmod +x scripts/*.sh
./scripts/phase5-validate.sh

# Provision
terraform -chdir=terraform init
terraform -chdir=terraform plan -out=phase5.tfplan
terraform -chdir=terraform apply phase5.tfplan

# Connect and build
gcloud container clusters get-credentials vertex-agent-lab --zone us-central1-a --project class-6-5-
tiqs
./scripts/build-phase5-images.sh
./scripts/install-falco.sh

# Deploy in fail-closed mode
./scripts/deploy-phase5.sh

# Verify
./scripts/smoke-test-metrics.sh
./scripts/test-phase5-governance.sh
./scripts/test-pipeline.sh

# Review and sign a pending approval when required
./scripts/review-phase5-approval.sh approve analyst@example.com "Approved after evidence review"
```

Operational handoff

Handoff item	Operational requirement
Owner	AI Platform / Cloud Security engineering
Primary runbook	docs/RUNBOOK.md
Runtime health	Pod probes, PodMonitoring, Cloud Monitoring, and Pub/Sub backlog alerts
Security evidence	Cloud Logging, BigQuery, Firestore state, release-evidence artifacts
Incident path	Inspect agent logs, Pub/Sub debug subscriptions, and the DLQ before replaying or modifying messages
Emergency stop	Set AUTOMATION_MODE and EXECUTION_MODE to disabled; rotate MCP credentials if needed
Change control	Pull request validation, signed release evidence, protected production environment, reviewed digest deployment

Appendix A: Major component summary

Component	Role
GKE and Workload Identity	Runs isolated Kubernetes workloads and maps service identities without static cloud keys.
Artifact Registry	Stores immutable container images and release digests.
Pub/Sub	Connects every security pipeline stage with retries and dead-letter handling.
Event Aggregator	Normalizes Falco, Trivy, Prowler, Cloud Logging, and SCC findings.
Observer Agent	Validates and enriches incoming findings.
Correlation Agent	Combines related findings into incidents and stores correlation state in Firestore.
IR Analyst Agent	Uses Gemini to assist explanation, confidence, severity, and response recommendation.
Governance Agent + OPA	Converts an AI recommendation into a deterministic authorization decision.
Cloud KMS + Approval Agent	Signs and validates human approval while enforcing expiry, binding, and replay controls.
Remediation Agent	Acts as the policy-enforcement point and independently verifies authorization.
MCP Gateway	Provides an nginx mTLS boundary that accepts only the Remediation Agent certificate.
MCP Server	Executes a small set of validated, RBAC-scoped Kubernetes tools.
Reporting Agent	Creates the final incident report and optional Slack/Jira delivery.
Firestore	Stores correlation, approval, and execution state.
BigQuery	Stores durable governance and audit evidence.

Component	Role
Managed Prometheus	Collects metrics and supports dashboarding and alerts.
Falco / Trivy / Prowler	Provide runtime, image/configuration, and cloud-posture findings.
GitHub Actions / WIF	Validates and releases the platform without a long-lived service-account key.
Syft / Cosign	Generate SBOMs, sign images, attach attestations, and verify release identity.

Real-world relevance

The project models how a modern enterprise security operations or AI platform team can use AI to accelerate incident triage without allowing the model to authorize itself. The same control objectives appear in real systems: event buses, agent runtimes, policy decision points, cryptographic approvals, SOAR-style execution workers, tool gateways, identity federation, observability, evidence retention, and signed software releases. A Fortune 500 implementation would add regional resilience, production PKI, admission-time trust enforcement, multi-tenancy, formal SLOs, enterprise SIEM/SOAR integration, red-team evaluation, and organizational change management.

Enterprise application and compensation outlook. In a Fortune 500 environment, this platform would operate between cloud-security telemetry and the company's incident-response ecosystem: it would normalize findings, correlate related signals, use AI to accelerate analyst decisions, enforce deterministic policy and signed approvals, execute only narrowly authorized Kubernetes remediation, and preserve evidence for audit, compliance, and post-incident review. In California's July 2026 market, current postings for comparable work include NVIDIA Senior AI Platform Engineer base ranges of $184,000-$287,500 at Level 4 and $224,000-$356,500 at Level 5, while Walmart principal cloud and AI platform roles list $143,000-$286,000; based on those published ranges, an experienced Senior AI Platform Engineer with a DevSecOps and cloud-security background could reasonably target approximately $185,000-$290,000 in base salary, and a Principal or Staff-level engineer approximately $225,000-$356,000 or more, with bonus and equity potentially raising total annual compensation into roughly the $250,000-$450,000+ range depending on employer, location, experience, and organizational scope.

Thirty-second elevator speech

Professional project introduction

I built a governed, multi-agent security automation platform on Google Kubernetes Engine. It collects and correlates cloud and runtime findings, then uses Gemini to help analysts understand incidents. The key distinction is that AI never directly controls the environment: OPA policy, Cloud KMS-signed human approval, Firestore replay controls, mTLS, MCP, and Kubernetes RBAC independently govern every action. I also implemented observability, vulnerability scanning, SBOM generation, keyless image signing, and immutable releases, demonstrating cloud engineering, Kubernetes, DevSecOps, security governance, and responsible AI integration.

Appendix B: Reusable Cursor AI Plan Mode prompt

```
You are acting as Cursor AI in PLAN MODE. Follow this workflow:

PHASE 1 - CLARIFICATION
Ask only the minimum technical questions needed to clarify scope, environment, existing files,
security requirements, success criteria, blockers, and dependencies. Do not write code.

PHASE 2 - RESEARCH
Inspect the Phase 5 GKE agentic-security repository, identify the exact files and components
affected, map dependencies, and identify risks or conflicts.

PHASE 3 - PLAN GENERATION
Create a detailed Markdown plan containing:
1. Overview and summary
```

```
2. Updated file structure
3. Step-by-step checklist with exact file references
4. Dependencies and prerequisites
5. Potential issues and mitigations
6. Testing strategy
7. Success criteria
8. Next steps
Also include time estimate, complexity rating, security/compliance considerations, multi-environment
differences, and rollback strategy.

PHASE 4 - REVIEW
Ask for approval and do not implement until the plan is approved.

PHASE 5 - IMPLEMENTATION
After approval, generate complete files, configuration, tests, documentation, and deployment
instructions. Preserve fail-closed defaults and do not enable controlled remediation without explicit
approval.

MY REQUEST:
[Describe the Phase 5 change, bug fix, deployment, refactor, or new feature here.]
```

Final review note

Current recommendation

Use this document as the planning and review record. Use README.md for the recruiter-facing overview and docs/RUNBOOK.md for exact deployment and operational commands. The next technical milestone is a live fail-closed deployment followed by governance, approval, evidence, and rollback validation.